4.5 Model Comparison: RMSE and Final Selection

2026-04-09

Goal

This section will compare forecasting performance across all methods using holdout RMSE and select the final model.

Methods included:

- Simple Rules (baseline models)
- Exponential Smoothing (ETS)
- ARIMA models
- ARMAX models

The primary evaluation metric is holdout RMSE, which reflects out-of-sample predictive accuracy.

RMSE Calculations

The following computations complete RMSE calculations for models derived from previous analysis. We will apply RMSE calculations to all candidate forecasting models to decide on a best-performing model with smallest RMSE.

Step 1: Setting up

```
suppressPackageStartupMessages({
  suppressWarnings(library(tidyverse))
  suppressWarnings(library(forecast))
})

agg <- readr::read_csv(
  "../../data/processed/store_type_a_train_aggregate_daily.csv",
  show_col_types = FALSE
) |>
  dplyr::mutate(Date = as.Date(Date)) |>
  dplyr::arrange(Date)

y_all <- agg$Sales

n_total <- length(y_all)
n_train <- floor(0.8 * n_total)

train_data <- y_all[1:n_train]
test_data  <- y_all[(n_train + 1):n_total]
holdout_idx <- (n_train + 1):n_total

rmse_vec <- function(a, p) sqrt(mean((a - p)^2, na.rm = TRUE))
```

Step 2: Pulling Simple and ETS Models

```r
simple_rmse <- readr::read_csv(
  "../../04_analysis/simple/results/simple_rules_aggregate_holdout_rmse_summary.csv"
) |>
  mutate(method = "Simple")

sales_ts <- ts(train_data, frequency = 7)
ets_model <- ets(sales_ts)
ets_fc <- forecast(ets_model, h = length(test_data))$mean

ets_rmse <- tibble(
  model = "ETS",
  holdout_rmse = rmse_vec(test_data, ets_fc),
  n_holdout = length(test_data),
  method = "ETS"
)
```

Step 3: ARIMA with Candidate Model Evaluation The ARIMA candidates below were selected to complement `auto.arima` with nearby low-order seasonal specifications informed by weekly seasonality (frequency = 7), ACF/PACF patterns from earlier diagnostics, and AIC-oriented model comparison.

```r
model1 <- auto.arima(sales_ts)
model2 <- Arima(sales_ts, order=c(4,0,1), seasonal=c(2,1,1))
model3 <- Arima(sales_ts, order=c(4,0,1), seasonal=c(1,1,0))
model4 <- Arima(sales_ts, order=c(2,0,1), seasonal=c(2,1,0))
model5 <- Arima(sales_ts, order=c(4,0,2), seasonal=c(2,1,0))

models <- list(
  ARIMA_auto = model1,
  ARIMA_m2 = model2,
  ARIMA_m3 = model3,
  ARIMA_m4 = model4,
  ARIMA_m5 = model5
)

arima_rmse_table <- purrr::imap_dfr(models, function(mod, name) {

  fc <- forecast(mod, h = length(test_data))$mean

  tibble(
    model = name,
    holdout_rmse = rmse_vec(test_data, fc),
    n_holdout = length(test_data),
    method = "ARIMA"
  )
}) |>
  arrange(holdout_rmse)

arima_best <- arima_rmse_table |> slice(1)
arima_best
```

```
## # A tibble: 1 x 4
##   model    holdout_rmse n_holdout method
##   <chr>           <dbl>     <int> <chr>
## 1 ARIMA_m2     1016497.       189 ARIMA
```

ARIMA Model Finalization

Based on the results above, the best ARIMA model out of all candidates is model 2 with RMSE = 1016497 (sale value). We will proceed with this model as a representative for the ARIMA method in the remaining analysis.

Step 4: ARMAX

```
armax_rmse <- readr::read_csv(
  "../../04_analysis/armax/results/armax_holdout_rmse_summary.csv",
  show_col_types = FALSE
)

armax_rmse <- armax_rmse |>
  mutate(method = "ARMAX")
```

Combining RMSE Results Across Explored Methods

```
rmse_all <- bind_rows(
  simple_rmse,
  ets_rmse,
  arima_best,
  armax_rmse
) |>
  mutate(model_full = paste(method, model, sep = " - ")) |>
  arrange(holdout_rmse)

rmse_all
```

```
## # A tibble: 7 x 6
##   model                      holdout_rmse n_holdout method train_aic model_full
##   <chr>                             <dbl>     <dbl> <chr>      <dbl> <chr>
## 1 ARMAX_Promo_Open_DOW_Month      399948.       189 ARMAX     21906. ARMAX - AR~
## 2 iid_by_period                  1013538.       189 Simple       NA  Simple - i~
## 3 ARIMA_m2                       1016497.       189 ARIMA        NA  ARIMA - AR~
## 4 ETS                            1042205.       189 ETS          NA  ETS - ETS
## 5 persistence_by_period          1665219.       189 Simple       NA  Simple - p~
## 6 global_average                 1784744.       189 Simple       NA  Simple - g~
## 7 persistence                    2516853.       189 Simple       NA  Simple - p~
```

Interpretation: RMSE Table

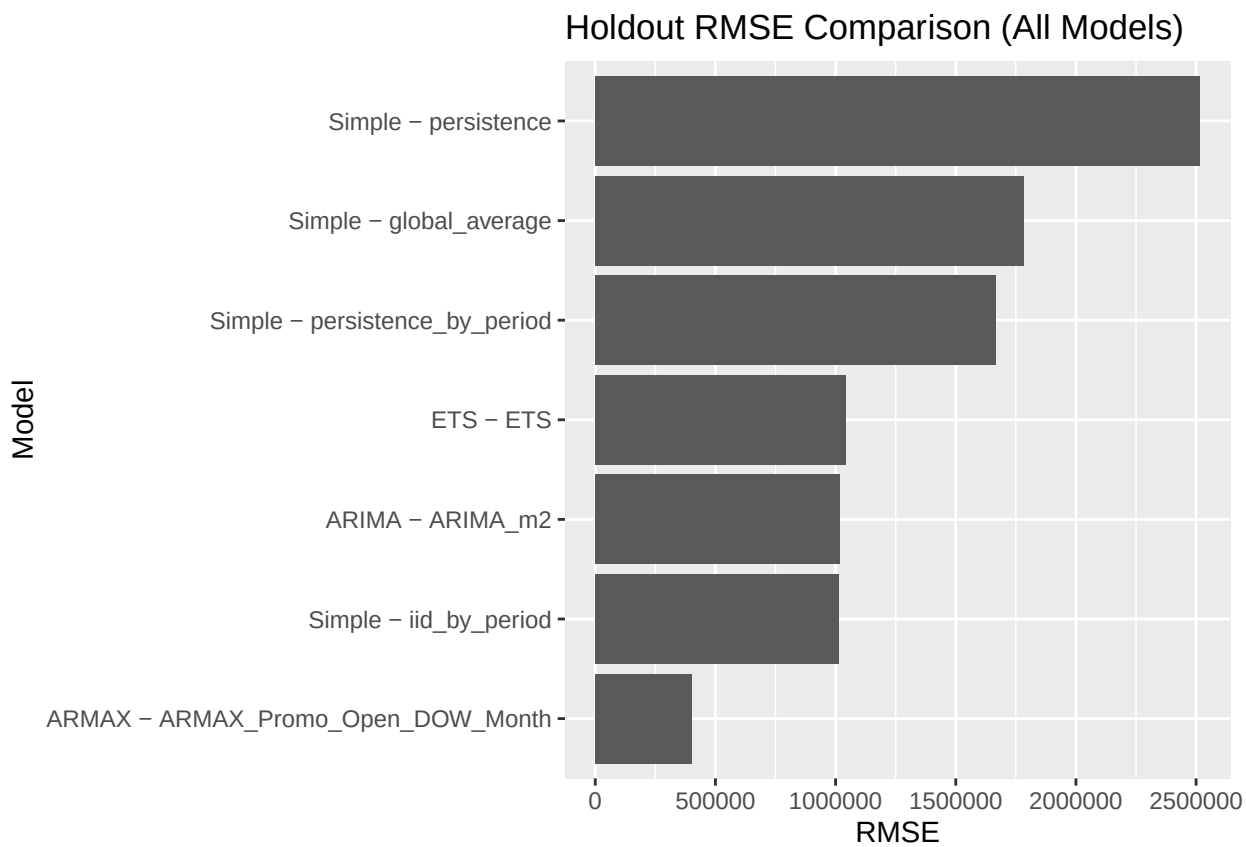
Overall the table above compares forecasting performance across all finalized models, where:

- Lower RMSE indicates better predictive accuracy.
- Baseline models such as persistence provide a benchmark for evaluating more complex approaches.
- Improvements over simple rules indicate that the more complex models successfully capture additional structure in the data.

Let's visualize these comparisons.

```
ggplot(rmse_all, aes(x = reorder(model_full, holdout_rmse), y = holdout_rmse)) +
  geom_col() +
  coord_flip() +
  labs(
```

```
    title = "Holdout RMSE Comparison (All Models)",
    x = "Model",
    y = "RMSE"
  )
```



Interpretation (RMSE Plot)

The bar chart above highlights differences in forecasting performance:

- ARMAX substantially outperforms all other methods by a wide margin, making it the clear best-performing model in this comparison.
- The large ARMAX gain is consistent with explanatory regressors (PromoShare, OpenStores, DayOfWeek, and Month) capturing demand drivers that univariate models cannot represent directly.
- ARIMA improves over most simple baselines but remains close to `iid_by_period`, indicating that day-of-week averaging alone already captures much of the weekly structure.
- The strong `iid_by_period` baseline result is plausible because averaging by day-of-week already captures much of the dominant weekly demand pattern, leaving limited additional gain for univariate ARIMA.

Final Model Selection

```
best_model <- rmse_all |> slice_min(holdout_rmse, n = 1)
best_model
```

```
## # A tibble: 1 x 6
##   model                      holdout_rmse n_holdout method train_aic model_full
##   <chr>                             <dbl>     <dbl> <chr>      <dbl> <chr>
## 1 ARMAX_Promo_Open_DOW_Month      399948.       189 ARMAX     21906. ARMAX - AR~
```

Interpretation: Final Model Choice

The selected model is the one with the lowest holdout RMSE (ARMAX model with RMSE = 399947.5) meaning the model demonstrates the best out-of-sample forecasting performance among the candidates.

Final Test Forecast

```
seasonal_frequency <- 7L
final_train <- readr::read_csv(
  "../../data/processed/store_type_a_train_aggregate_daily.csv",
  show_col_types = FALSE
) |>
  mutate(
    Date = as.Date(Date),
    PromoShare = if_else(OpenStores > 0, PromoStores / OpenStores, 0),
    DayOfWeek = factor(DayOfWeek, levels = 1:7),
    Month = factor(Month, levels = 1:12)
  ) |>
  arrange(Date)

final_test <- readr::read_csv(
  "../../data/processed/store_type_a_test_aggregate_daily.csv",
  show_col_types = FALSE
) |>
  mutate(
    Date = as.Date(Date),
    PromoShare = if_else(OpenStores > 0, PromoStores / OpenStores, 0),
    DayOfWeek = factor(DayOfWeek, levels = 1:7),
    Month = factor(Month, levels = 1:12)
  ) |>
  arrange(Date)

colnames(final_test)
```

```
##  [1] "Date"               "OpenStores"         "PromoStores"
##  [4] "PromoShare"         "SchoolHolidayShare" "DayOfWeek"
##  [7] "Year"               "Month"              "IsoWeek"
## [10] "IsWeekend"
```

```
xreg_train <- model.matrix(
  ~ PromoShare + OpenStores + DayOfWeek + Month,
  data = final_train
)[, -1]

xreg_test <- model.matrix(
  ~ PromoShare + OpenStores + DayOfWeek + Month,
  data = final_test
)[, -1]

y_train_full <- ts(final_train$Sales, frequency = seasonal_frequency)

final_armax_model <- auto.arima(
  y_train_full,
  xreg = xreg_train,
  seasonal = TRUE,
```

```r
  allowmean = FALSE,
  allowdrift = FALSE,
  stepwise = TRUE,
  approximation = TRUE
)

h <- nrow(final_test)

final_forecast <- forecast(
  final_armax_model,
  xreg = xreg_test,
  h = h
)$mean

# Enforce non-negative sales
final_forecast <- pmax(0, as.numeric(final_forecast))

test_target_col <- intersect(c("Sales", "sales"), colnames(final_test))

if (length(test_target_col) == 1) {
  final_results <- tibble(
    Date = final_test$Date,
    actual = final_test[[test_target_col]],
    predicted = final_forecast
  )

  final_test_rmse <- sqrt(mean(
    (final_results$actual - final_results$predicted)^2,
    na.rm = TRUE
  ))
} else {
  final_results <- tibble(
    Date = final_test$Date,
    predicted = final_forecast
  )

  final_test_rmse <- NA_real_
  message("No actual sales column found in final_test; test RMSE cannot be computed from this file.")
}

head(final_results)
```

```
## # A tibble: 6 x 2
##   Date       predicted
##   <date>         <dbl>
## 1 2015-08-01  2515834.
## 2 2015-08-02   288768.
## 3 2015-08-03  3487401.
## 4 2015-08-04  2847204.
## 5 2015-08-05  2554873.
## 6 2015-08-06  2526256.
```

```r
final_test_rmse
```

```
## [1] NA
```

Interpretation: Test Forecast

The final model is used to generate forecasts for a completely unseen test feature set. In the current aggregate test file, actual sales are not available, so a true test RMSE cannot be computed here. Holdout RMSE remains the valid basis for model comparison in this notebook; test RMSE requires a labeled test file with actual sales.